

C++ 面试高频知识点整理

1. C++ 的四大类型转换及区别

- `static_cast`: 编译期类型转换, 常规转换 (如 `int` 转 `float`)。
- `dynamic_cast`: 运行时类型检查, 多用于多态。
- `const_cast`: 去除或添加 `const` 属性。
- `reinterpret_cast`: 底层位级别强制转换, 危险, 打破类型安全。

2. 智能指针原理及类型

- `shared_ptr`: 引用计数, 最后一个销毁时释放资源。
- `weak_ptr`: 观察者指针, 解决循环引用。
- `unique_ptr`: 独占所有权, 不可拷贝, 只能转移。

3. RAII 原理与作用

- 资源获取即初始化, 将资源管理与对象生命周期绑定。
- 自动释放资源, 防止泄漏, 异常安全。

4. 深拷贝与浅拷贝区别

- 浅拷贝: 复制指针地址, 多指针指向同一内存。
- 深拷贝: 复制内容本身, 分配新内存, 防止重复释放。

5. C++11 三大特性

- 移动语义 (右值引用、`std::move`)
- 智能指针
- Lambda 表达式

6. 左值/右值与引用

- 左值: 有名字、可寻址。
- 右值: 临时对象, 不可寻址。
- 左值引用: 绑定左值。
- 右值引用: 绑定右值, 用于资源转移。

7. STL 容器原理 (vector, map)

- `vector`: 动态数组, 扩容通常为2倍。
- `map`: 红黑树实现, $O(\log N)$ 查找。

8. 内存对齐

- 提高 CPU 访问效率, 结构体成员按特定字节对齐。
- 避免跨字节访问带来性能损失。

9. 多态原理与虚函数表

- 基类指针指向子类对象，通过虚函数实现动态绑定。
- 对象中有虚表指针（vptr），指向虚函数表（vtable）。

10. 虚函数与纯虚函数区别

- 虚函数：可被子类重写，父类有默认实现。
- 纯虚函数：`=0`，子类必须重写，父类为抽象类。

11. C++ 对象模型

- 对象有自己的数据成员。
- 成员函数存在代码段，不属于对象本身。
- 有虚函数时，对象有一份 vptr 指针。

12. 拷贝构造与赋值运算符区别

- 拷贝构造：用已有对象初始化新对象。
- 赋值运算符：将已有对象的值赋给已存在对象。

13. 移动构造函数

- 接收右值，偷取资源而非复制，提高性能。
- 触发：返回局部变量、临时对象初始化等。

14. Lambda 表达式底层原理

- 编译器生成匿名类，重载 `operator()`。
- 实质为轻量仿函数对象。

15. 函数模板与类模板

- 函数模板：支持泛型函数。
- 类模板：支持泛型数据结构（如 `vector`）。

16. 异常机制原理

- `try-catch` 捕获异常，抛出异常时栈展开，自动析构清理资源。

17. C++ 内存管理：new/delete 与 malloc/free 区别

- `new/delete`：调用构造/析构函数，类型安全。
- `malloc/free`：仅分配/释放内存，不调用构造/析构。

18. 静态成员变量与静态成员函数

- 静态变量：类共享一份。
- 静态函数：不能访问非静态成员，无 `this` 指针。

19. volatile 关键字

- 告诉编译器变量可能被外部修改，防止优化。

20. C++ 线程库基础

- `std::thread` 创建线程。
- `std::mutex` 实现互斥锁。
- `std::condition_variable` 实现线程同步。

21. 宏定义、内联函数、模板区别与应用场景

- 宏定义：预处理替换，无类型检查，易错。
- 内联函数：编译期展开，有类型检查，适合小函数。
- 模板：泛型编程，编译期生成不同类型代码。

22. 模板代码体积膨胀解决方法

- 精简模板逻辑，特化常见场景，类型擦除，显式实例化。

23. 友元 (friend) 使用场景与缺点

- 用于操作符重载、类间高度耦合时访问私有成员。
- 缺点：破坏封装性，增加耦合，降低可维护性。

24. 内存泄漏、越界、野指针

- 内存泄漏：堆内存未释放。
- 内存越界：非法访问内存（如数组越界）。
- 野指针：指向已释放或未初始化内存。

25. new 和 malloc 区别

- new：C++ 专有，调用构造函数，类型安全，异常处理。
- malloc：C 库函数，仅分配内存，不调用构造函数，需强转类型。
- 释放：new/delete 配对，malloc/free 配对。